- **Correctness:** your mapping is correct, but CNL-P spans both requirements and design, and IRs must include evaluation + binding planes.
- **Completeness:** add versioning, waivers, evaluation/regression, observability, portability.
- **Practicality:** feasible if you focus on a few skill archetypes first and treat normalization as “draft + review” rather than fully automatic.

If you want, I can turn this into a **milestone-by-milestone deliverable checklist** (artifacts, APIs, tests) for the first two target skills so it reads like an engineering plan you can execute.

When analyzing Anthropic's skill, a key finding is:
"... your pipeline covers “procedural + artifact” skills extremely well, butneeds “capability + evaluation + interaction” to cover the whole repo. ..."

InteractionIR is necessary for:
"InteractionIR- elicitation schema (required questions, optional questions, stop conditions)- progressive disclosure pointers (when to consult references/examples)- crucial for comms/coauthoring/brand-guideline skills"

We analyze that:
" ... —but for some skills, WorkflowIR/ArtifactIR will be “thin” and PolicyIR will be mostly soft."

An example is Anthropic's internal-comms skill (https://github.com/anthropics/skills/tree/main/skills/internal-comms), as summarized
"3) Guidance / templates: internal-comms (interaction + information gathering)
WorkflowIRclassify_request → select_template → gather_sources → draft → revise...but most steps are conversational."

We analyze that
"For this category, the missing piece is not more schema or more rego—it’s a first-class InteractionIR:- required

questions / missing context prompts- tool-query plans (“search Slack channels with high reactions”, “find exec emails”)- progressive drafting loopThe skill literally encodes a “load guideline file then follow it” recipe.That is closer to compiling into a dialogue policy + retrieval plan than into a deterministic transformer."

We also observe that:
"CNL-P struggles if you expect it to fully replace: - tacit heuristics (“good taste UI”, “avoid AI slop”, “write a compelling comms draft”), - open-ended creativity, - social/organizational context (internal comms, brand tone), - tool-specific nuance embedded in prose. CNL-P can still host these, but they should compile to: - soft rules / rubrics, - interaction plans (questions to ask, evidence to request), - review gates."

We determine that:
"Leave to LLM execution when:- the task is fundamentally creative/interpretive,- correctness depends on human preference/context,- inputs are incomplete and require negotiation,- evaluation is holistic and lacks reliable sensors.The generation remains LLM-led."

Two specific scenarios as follows:
"B. Underspecified intent and requirement discoveryMany skills begin with “figure out what the user really wants.”That requires:- asking questions- resolving ambiguity- negotiating tradeoffsYou can operationalize the interaction policy (what to ask), but the actual understanding and negotiation is still LLM-driven."

"C. Open-world reasoning and missing factsWhenever the skill requires facts that aren’t present in the environment:- business context- organizational policy exceptions- implicit constraintsYou can operationalize: “don’t assume; ask.”But the actual completion depends on human/LLM knowledge acquisition."

Using this Anthropic's internal-comms skill as an example:
Although it defines a workflow classify_request →

select_template → gather_sources → draft → revise, it is a very high-level one more like a workflow guide than an actual interaction plan. As pointed out in the quote "but most steps are conversational.", the actual human-AI interaction would unfold as it goes, involving flexible rounds of interactions in one step, iteration between steps and back and forth, revisions of collected information from previous steps. There is no way to pre-define the actual interaction flow.

In our discussion, we see an example: "INTERACTION:REQUIRE_INPUTS [...]ASK:- IF missing(inputs.X) THEN ASK "Question?"- IF ambiguous(Y) THEN ASK_CHOICE "Pick one" OPTIONS [...]STOP:- IF user_declines THEN STOP_WITH "..."

But I think this interaction plan is too rigid. However, I do not want the interaction plan to be completely free form either. The challenge is how to balance the controllability and flexibility.

Systematically understand these quotes from our discussion and my concerns, and propose practical approach:
1. the InteractionIR design;
2. the strategy and process to compile skill guidelines/steps descriptions into interaction plan/IR
3. how to operatioinalize the interaction plan

You're pointing at the real crux: **interaction is the least "compilable" plane**, because it's inherently adaptive, yet you still want *some* controllability (no hallucinated assumptions, structured info capture, safe tool usage, good user experience). The right approach is **not** to predefine a rigid dialogue tree, but to compile skills into a **policy-guided interaction controller**: a small set of *constraints*, *obligations*, and *decision points* that shape an otherwise flexible LLM conversation.

Below is a practical approach in three parts:

1. InteractionIR design
2. How to compile skill prose into InteractionIR
3. How to operationalize (execute + enforce) InteractionIR in runtime

1) Practical InteractionIR design: controllable constraints, flexible execution

Core design principle

InteractionIR should not be a script of the conversation.
It should be:

- a **stateful information model** (what we need to know),
- a set of **interaction obligations** (when must we ask / confirm / stop),
- **tool-use policies** (what can be queried, when, and how results must be presented),
- and **loop controls** (how to iterate without thrashing).

Think "controller" rather than "flowchart".

InteractionIR v1: components

A) InfoSpec: the "requirements contract" for the conversation

This is the backbone. It defines the *slots* the conversation must fill (or explicitly leave unknown with justification).

- `slots` : required / optional / derived
- `types` : enum/text/date/number/URI/list
- `constraints` : allowed ranges, mutual exclusivity
- `provenance` : how a slot can be filled (user answer / tool result / inference forbidden)

Example slots for internal comms:

- `comms_type` (enum: newsletter, announcement, update, exec-brief…)
- `audience` (enum/list)
- `time_window` (date range)
- `channels` (slack/email/docs)
- `tone` (enum: neutral, upbeat, urgent)
- `must_include` (list)
- `length_budget` (target bullets/words)
- `approver` (optional)
- `sources[]` (evidence links/snippets)

B) Guardrails: "don't assume; ask" and safety constraints

These are enforceable policies:

- forbidden assumptions for certain slots
- required confirmation before sensitive actions

- stop conditions (insufficient info, user declines, conflict)

Example:

- "Do not invent links or quote unseen sources."
- "If requesting company-internal info, require tool evidence or user-provided text."

C) Dialogue Acts: constrained move set
Instead of hardcoding conversation steps, define a **small taxonomy of allowed dialogue actions**:

- `ASK_SLOT(slot_id)`
- `ASK_CLARIFY(slot_id, options)`
- `PROPOSE_DRAFT(version_id)`
- `REQUEST_SOURCES(source_type, query_plan)`
- `SUMMARIZE_UNDERSTANDING(slot_snapshot)`
- `CONFIRM_DECISION(decision_point)`
- `SUGGEST_OPTIONS(tradeoff)`
- `STOP(reason)`

This yields flexibility (LLM chooses which act) while still being controllable (runtime checks validity).

D) Policies over acts (when must we do what)
Policies are written as **constraints**, not scripts:

- **Obligations**: "Before producing final output, ensure required slots are filled or explicitly marked unknown."
- **Triggers**: "If ambiguity detected, must ask clarify within N turns."
- **Budgets**: limit number of clarifying questions per iteration to avoid interrogation.
- **Iterative loop rules**: how to revisit slots when new evidence arrives.

E) Tool-query plans as templates, not fixed sequences
For internal-comms, "gather_sources" is huge and variable. InteractionIR should encode tool use as:

- `source_recipes` (Slack search patterns, email search operators, calendar windows)
- `ranking/coverage goals` (diversity of sources, recency)
- `stopping criteria` (e.g., enough items found, or diminishing returns)
- `evidence extraction contract` (each bullet must cite source snippet/link if sourced)

This is controllable without being rigid.

F) Checkpoints

Add checkpoints like you did for workflow:

- `checkpoint: readiness_to_draft`
- `checkpoint: readiness_to_publish`
- `checkpoint: revision_complete`

At checkpoints the policy engine evaluates:

- slot completeness
- evidence presence
- constraints met
  and decides: allow / require questions / require review.

---

2) Compile skill guidelines into InteractionIR: strategy and process

Your concern is correct: skills like internal-comms have high-level steps and a lot of tacit judgement. The compilation cannot be purely syntactic. The practical compiler must do **semantic extraction + normalization + conservative defaults**.

A) What to extract from skill text (and examples)

For internal-comms-like skills, the richest signal is often in:

- example templates (newsletter format, bullet count, “lots of links”)
- “what to do first” language
- source suggestions (“search Slack channels with high reactions”)
- formatting constraints

So compilation should include **examples as first-class sources**, not just the main SKILL.md.

B) Compilation pipeline for InteractionIR

Step 1: Clause mining & classification

Extract clauses and tag them using your operationalizability rubric, but with interaction-specific labels:

- `Info requirement` (“need audience”)
- `Clarification trigger` (“if type unknown, ask”)
- `Source requirement` (“include links, cite sources”)
- `Style constraint` (“bullet format, concise, high signal”)
- `Tool plan hint` (“search Slack high reactions”)
- `Stop condition` (“if insufficient sources, ask user to provide”)

Step 2: Build InfoSpec (slots) from requirements
Turn “must ask / must decide” content into slots.

Heuristic rules:

- any recurring field in templates → slot
- anything that changes structure/tone → slot
- anything that changes tool queries (time range, channel) → slot
- anything high risk (audience, confidentiality) → slot

Mark provenance:

- **must come from user or tools** vs **may be inferred**.

Step 3: Build Tool Recipes from “gather_sources”
Convert tool guidance into parameterized recipes:

Example recipe object:

- `name: slack_top_updates`
- `tool: slack.search`
- `query_template: "in:#announcements after:{start} before:{end} has:link"`
- `ranking: reactions desc`
- `stop_when: found>=N or pages>=k`

Even if connectors differ across orgs, this is still portable as an abstract recipe.

Step 4: Generate policies (obligations/triggers/budgets)
Translate “don’t assume; ask” into concrete policies:

- `required_slots` must be filled before draft
- `max_questions_per_turn`
- `ask_clarify` triggers when slot values conflict
- `evidence_required` for externally-sourced facts

Step 5: Extract iteration loop semantics
From “draft → revise”, compile:

- `revise_mode` : accept edits, then re-check constraints
- `revision budget` : number of revision cycles before asking user to prioritize
- `diff summarization` : show what changed between drafts

Step 6: Emit InteractionIR + “uncertainty markers”
Where the skill is underspecified:

- emit `slot.optional` rather than required
- emit `policy.require_review` if critical missing
- include `TODO: needs domain profile` markers
  This prevents overfitting.

C) Key design choice: compile *constraints*, not *dialogue*

You're right that trying to compile into a fixed ASK/ANSWER tree is too rigid. The compiler should output:

- **slot model + triggers + tool recipes + checkpoints**
  and leave the conversational micro-flow to the LLM under constraints.

---

3) Operationalize the interaction plan: execute + enforce without rigidity

This is the runtime architecture.

A) Interaction runtime as a controller loop

Each user/agent turn runs:

1. **Update state**
   Extract slot values from user message/tool results (with provenance).
2. **Policy check** (InteractionPolicy / Rego-like)
   Evaluate:
- missing required slots?
- conflicts?
- exceeding question budget?
- evidence requirements met?
3. **Action selection**
   If policy says "must ask/clarify", the system constrains the LLM:
- "You must perform one of these dialogue acts next: ASK_SLOT(audience) or ASK_CHOICE(comms_type)..."
  If policy allows drafting, the LLM may draft.
4. **Tool orchestration** (optional)
   If the next act is `REQUEST_SOURCES`, the runtime executes tool recipes and stores evidence.
5. **Checkpoint gating**
   Before "publish/finalize", enforce checkpoint conditions.

B) How to keep flexibility

Key mechanisms to avoid rigidity:

1. **Soft constraints + budgets rather than fixed sequences**
- "ask at most 2 questions per turn"
- "prefer asking highest-information-gain slot first"
- "if missing >3 required slots, summarize what's missing and ask user to prioritize"
2. **LLM chooses *which* slot to ask within allowed set**
   Policy produces an allowed action set, not a single mandated question.
3. **Progressive disclosure**
- default ask only what's necessary to move forward
- allow "draft with assumptions" only if assumptions are explicitly marked and user confirms
4. **Evidence-first behavior**
   If connectors exist, prefer tool evidence over questioning the user for everything.

C) Enforcement surfaces

What can the runtime enforce deterministically?

- **No-invention policy**: any factual claim must be supported by user input or tool evidence; otherwise mark as assumption.
- **Required slots at checkpoints**: deny publish if missing.
- **Confidentiality constraints**: block forbidden actions/tools.
- **Traceability**: store slot provenance and evidence URIs.

D) How it interfaces with your existing compiler stack

- InteractionIR compiles to:
  - InteractionPolicy (Rego or a small rule engine)
  - Slot schema (DataIR)
  - Tool recipe templates (Bindings/CapabilityIR)
  - Checkpoints (same policy-decision API pattern)

So it plugs into your same "policy decision API" concept, just with a different plane of concerns.

A concrete InteractionIR sketch for internal-comms

Here is a minimal InteractionIR fragment illustrating the above design:

```
{
  "interaction": {
    "id": "int.internal_comms.v1",
    "slots": [
      { "id": "comms_type", "type": "enum", "values": ["newsletter","annou
      { "id": "audience", "type": "text", "required_for": ["publish"], "pr
      { "id": "time_window", "type": "date_range", "required_for": ["gathe
      { "id": "length_budget", "type": "int", "default": 25, "constraints"
    ],
    "policies": [
      { "id": "no_invent_links", "type": "deny", "when": "draft.contains_l
      { "id": "ask_if_missing", "type": "obligation", "when": "checkpoint
      { "id": "question_budget", "type": "budget", "max_questions_per_turn
    ],
    "tool_recipes": [
      { "id": "slack_announcements", "tool": "slack.search", "query": "in:
    ],
    "checkpoints": [
      { "id": "draft", "requires_slots": ["comms_type"] },
      { "id": "publish", "requires_slots": ["comms_type","audience"], "req
    ]
  }
}
```

This is flexible: it doesn't dictate turn-by-turn conversation, but it *does* enforce "don't assume; ask", evidence requirements, and question budgets.

Summary recommendations (practical)

1) InteractionIR design

- Model **slots + provenance + checkpoints + policies + tool recipes + budgets**

JSON

2) Compilation strategy

- Extract **info requirements** from templates/examples
- Extract **tool search heuristics** into parameterized recipes
- Compile **obligations** ("must ask if missing") not fixed questions
- Use rubric outputs to decide what becomes a slot, a policy, or a rubric item

3) Operationalization

- Run an **interaction controller loop** each turn:

    - update slot state → evaluate policy → restrict allowed dialogue acts → execute tools if needed → checkpoint gate
- Enforce “no invention”, required slots, evidence requirements, and budgets deterministically.
- Leave micro-level phrasing and negotiation to the LLM within those constraints.

If you want, I can take the actual `internal-comms` newsletter example and produce a more complete InteractionIR: slots, tool recipes, checkpoint gates, and the mapping to a policy-decision API response format (allowed acts + required evidence + next recommended queries).

Do so "take the actual internal-comms newsletter example and produce a more complete InteractionIR: slots, tool recipes, checkpoint gates, and the mapping to a policy-decision API response format (allowed acts + required evidence + next recommended queries)."

Thought for 25s

Below is a **more complete InteractionIR** derived specifically from Anthropic’s `internal-comms` **company newsletter** guideline (`examples/company-newsletter.md`), plus a concrete **policy-decision API mapping** that returns:

- allowed dialogue acts,
- required evidence,
- next recommended tool queries / questions.

I’m grounding the slots/constraints/tool recipes in the actual guideline content: bullet count (~20–25), 1–2 sentences per bullet, “we” tense, lots of links, source types (Slack, email, calendar, docs, external press), sectioning/clustering, prioritization/avoid list.

GitHub +1

---

1) InteractionIR for `company-newsletter`

1.1 Core idea

Instead of hardcoding a conversation flow, this InteractionIR defines:

- **InfoSpec (slots)**: what must be known to proceed
- **Tool recipes**: parameterized retrieval plans (Slack/email/calendar/docs/press)
- **Checkpoints**: readiness gates for gather → draft → publish
- **Policies**: evidence, “don’t assume”, budgeting, coverage

1.2 InteractionIR (JSON)

```
{
  "interaction": {
    "id": "int.internal_comms.company_newsletter.v1",
    "title": "Company newsletter interaction controller",
    "source_guideline": "skills/internal-comms/examples/company-newsletter
    "slots": [
      {
        "id": "time_window",
        "type": "date_range",
        "required_for": ["gather_sources", "draft"],
        "default_strategy": "ask_user",
        "notes": "Guideline suggests past week/month; choose explicitly."
      },
      {
        "id": "distribution_channels",
        "type": "set_enum",
        "values": ["slack", "email"],
        "required_for": ["draft"],
        "default": ["slack", "email"],
        "constraints": { "min_size": 1 }
      },
      {
        "id": "bullet_target_count",
        "type": "int_range",
        "required_for": ["draft"],
        "default": { "min": 20, "max": 25 },
        "constraints": { "min": 10, "max": 40 },
        "provenance": "guideline"
      },
      {
        "id": "bullet_sentence_limit",
        "type": "int_range",
        "required_for": ["draft"],
        "default": { "min": 1, "max": 2 },
        "provenance": "guideline"
      },
      {
        "id": "voice_tense",
        "type": "enum",
        "values": ["we"],
        "required_for": ["draft"],
```

```
      "default": "we",
      "provenance": "guideline"
    },
    {
      "id": "sections_strategy",
      "type": "enum",
      "values": ["company_announcements_progress_leadership_social", "cu
      "required_for": ["draft"],
      "default": "company_announcements_progress_leadership_social"
    },
    {
      "id": "coverage_dimensions",
      "type": "set_enum",
      "values": ["product", "go_to_market", "finance", "recruiting", "ex
      "required_for": ["draft"],
      "default": ["internal_news", "external_news"]
    },
    {
      "id": "must_include_topics",
      "type": "list_text",
      "required_for": [],
      "default": []
    },
    {
      "id": "must_exclude_topics",
      "type": "list_text",
      "required_for": [],
      "default": ["overly granular team updates", "small-group-only info
      "provenance": "guideline"
    },
    {
      "id": "sources_available",
      "type": "set_enum",
      "values": ["slack", "email", "calendar", "docs", "external_press",
      "required_for": ["gather_sources"],
      "default_strategy": "detect_capabilities"
    }
  ],
  "evidence_types": [
    {
      "id": "ev.slack_message",
      "fields": ["channel", "message_url", "author", "timestamp", "react
    },
    {
      "id": "ev.email",
      "fields": ["from", "subject", "date", "message_url_or_id", "snippe
    },
    {
      "id": "ev.calendar_event",
      "fields": ["title", "date", "attendee_count", "description_snippet
    },
    {
```

```
      "id": "ev.doc",
      "fields": ["title", "doc_url", "owner_or_author", "published_date"
    },
    {
      "id": "ev.press",
      "fields": ["title", "url", "publisher", "date"]
    }
  ],
  "policies": [
    {
      "id": "p.bullet_count_target",
      "type": "soft_guardrail",
      "when": "checkpoint == 'draft'",
      "require": "draft.bullets_count between bullet_target_count.min an
      "on_fail": { "action": "require_revision", "message": "Adjust to ~
    },
    {
      "id": "p.bullet_length",
      "type": "soft_guardrail",
      "when": "checkpoint == 'draft'",
      "require": "each_bullet.sentences <= bullet_sentence_limit.max",
      "on_fail": { "action": "require_revision", "message": "Keep bullet
    },
    {
      "id": "p.we_voice",
      "type": "soft_guardrail",
      "when": "checkpoint == 'draft'",
      "require": "draft.uses_we_voice == true",
      "on_fail": { "action": "require_revision", "message": "Use 'we' te
    },
    {
      "id": "p.links_presence",
      "type": "medium_evidence_requirement",
      "when": "checkpoint in ['draft','publish']",
      "require": "evidence.links_count >= 8",
      "on_fail": { "action": "require_more_sources", "message": "Include
    },
    {
      "id": "p.prioritization",
      "type": "soft_guardrail",
      "when": "checkpoint == 'draft'",
      "require": "draft.prioritizes_company_wide_impact == true",
      "on_fail": { "action": "require_revision", "message": "Prioritize
    },
    {
      "id": "p.no_invention",
      "type": "hard_guardrail",
      "when": "checkpoint in ['draft','publish']",
      "require": "all_links_and_claims_have_provenance == true",
      "on_fail": { "action": "deny_publish", "message": "Do not invent l
    },
    {
```

```
      "id": "p.question_budget",
      "type": "budget",
      "max_questions_per_turn": 2,
      "max_rounds_without_progress": 2,
      "fallback": "propose_draft_with_marked_assumptions"
    }
  ],
  "tool_recipes": [
    {
      "id": "q.slack.high_signal_announcements",
      "source": "slack",
      "intent": "Find high-signal messages in large channels with strong
      "params": ["time_window", "channels_hint"],
      "query_template": "in:(#announcements OR #company OR #all-hands) a
      "ranking": ["reactions_count desc", "thread_replies_count desc"],
      "stop_when": { "min_items": 10, "max_pages": 3 },
      "evidence_type": "ev.slack_message"
    },
    {
      "id": "q.email.exec_companywide",
      "source": "email",
      "intent": "Find company-wide announcements from executives",
      "params": ["time_window", "exec_senders_hint"],
      "query_template": "from:({exec_senders}) newer_than:{days}d",
      "ranking": ["date desc"],
      "stop_when": { "min_items": 5 },
      "evidence_type": "ev.email"
    },
    {
      "id": "q.calendar.large_meetings",
      "source": "calendar",
      "intent": "Find all-hands / large attendee meetings and linked doc
      "params": ["time_window"],
      "query_template": "attendees_count>=200 OR title:(All Hands OR Tow
      "ranking": ["attendee_count desc", "date desc"],
      "stop_when": { "min_items": 3 },
      "evidence_type": "ev.calendar_event"
    },
    {
      "id": "q.docs.recent_high_attention",
      "source": "docs",
      "intent": "Find high-attention docs (vision, quarterly plans, exec
      "params": ["time_window"],
      "query_template": "modified_after:{start_minus_14d} OR created_aft
      "ranking": ["attention_signal desc", "published_date desc"],
      "stop_when": { "min_items": 5 },
      "evidence_type": "ev.doc"
    },
    {
      "id": "q.press.recent_mentions",
      "source": "external_press",
      "intent": "Collect notable press mentions within the time window",
```

```
          "params": ["time_window", "company_name"],
          "query_template": "{company_name} press after:{start} before:{end}
          "ranking": ["publisher_tier", "date desc"],
          "stop_when": { "min_items": 2 },
          "evidence_type": "ev.press"
        }
      ],
      "checkpoints": [
        {
          "id": "gather_sources",
          "requires_slots": ["time_window", "sources_available"],
          "passes_if": "true"
        },
        {
          "id": "draft",
          "requires_slots": ["time_window", "distribution_channels", "bullet
          "requires_evidence": ["evidence.links_count>=8"],
          "passes_if": "policies.p.no_invention == true"
        },
        {
          "id": "publish",
          "requires_slots": ["time_window", "distribution_channels"],
          "requires_evidence": ["evidence.links_count>=8"],
          "passes_if": "policies.p.no_invention == true"
        }
      ]
    }
  }
```

Where this comes from in the guideline

- ~20–25 bullet points; sent via Slack and email. GitHub
- Lots of links from docs, Slack announce channels/executives, company-wide emails, calendar attachments, external press. GitHub
- 1–2 sentences per bullet; “we” tense; “we did this…” phrasing. GitHub
- Break into sections/clusters; highlight different areas; prioritize company-wide impact + leadership announcements; avoid granular/duplicate info. GitHub +1

---

2) Mapping to a policy-decision API response format

2.1 Decision API: inputs (runtime state)

At each turn (or at checkpoints), the runtime collects:

- slot state (what is filled and provenance),
- evidence counts (links, sources),
- tool availability (Slack/email/calendar/docs connectors),

- current stage (`gather_sources` / `draft` / `publish`).

2.2 Decision API response schema (shape)

```
{
  "decision": "allow | require_input | require_tool_query | require_review
  "allowed_acts": ["ASK_SLOT", "ASK_CHOICE", "REQUEST_SOURCES", "PROPOSE_D
  "required_evidence": [{ "type": "...", "constraint": "..." }],
  "next_recommended_queries": [{ "recipe_id": "...", "params": { } }],
  "questions_to_ask": [{ "slot": "...", "prompt": "...", "options": [] }],
  "notes": ["..."]
}
```

2.3 Example A: initial user request is vague

JSON

Decision response

```
{
  "decision": "require_input",
  "allowed_acts": ["ASK_SLOT", "ASK_CHOICE", "STOP"],
  "required_evidence": [],
  "next_recommended_queries": [],
  "questions_to_ask": [
    {
      "slot": "time_window",
      "prompt": "Is this newsletter covering the past week or the past mon
      "options": ["past week", "past month", "custom range"]
    },
    {
      "slot": "must_include_topics",
      "prompt": "Any must-include announcements or topics you already know
      "options": []
    }
  ],
  "notes": [
    "Need time window to gather sources and shape newsletter per guideline
    "Question budget: ask max 2 questions this turn."
  ]
}
```

2.4 Example B: time window known, tools available → gather sources

Assume connectors exist; you have `time_window` and want evidence.

```
{
  "decision": "require_tool_query",
  "allowed_acts": ["REQUEST_SOURCES", "ASK_SLOT", "STOP"],
```

JSON

```
  ],
  "next_recommended_queries": [
    {
      "recipe_id": "q.slack.high_signal_announcements",
      "params": { "time_window": "2026-02-17..2026-02-24", "channels_hint"
    },
    {
      "recipe_id": "q.email.exec_companywide",
      "params": { "time_window": "2026-02-17..2026-02-24", "exec_senders_h
    },
    {
      "recipe_id": "q.calendar.large_meetings",
      "params": { "time_window": "2026-02-17..2026-02-24" }
    },
    {
      "recipe_id": "q.docs.recent_high_attention",
      "params": { "time_window": "2026-02-17..2026-02-24" }
    }
  ],
  "questions_to_ask": [],
  "notes": [
    "Guideline: include lots of links; prioritize leadership announcements
    "Stop when enough items found or diminishing returns."
  ]
}
```

2.5 Example C: evidence gathered → allow drafting, but enforce constraints

Assume evidence has enough links and provenance.

```
{
  "decision": "allow",
  "allowed_acts": ["PROPOSE_DRAFT", "ASK_SLOT", "REQUEST_SOURCES"],
  "required_evidence": [
    { "type": "provenance", "constraint": "no invented links; each bullet
  ],
  "next_recommended_queries": [
    {
      "recipe_id": "q.press.recent_mentions",
```

JSON

```
    }
  ]
```

JSON

```
      "slot": "sections_strategy",
      "prompt": "Prefer the default sections (Announcements / Progress / L
      "options": ["default", "custom clusters"]
    }
  ],
  "notes": [
    "Draft should be ~20–25 bullets, 1–2 sentences each, 'we' voice.",
    "Avoid duplicates and overly granular team updates."
  ]
}
```

2.6 Example D: attempt to publish but no evidence → block publish, request user input

If tools unavailable or evidence insufficient, you require user-provided items.

```
{
  "decision": "require_review",
  "allowed acts": ["ASK SLOT". "REQUEST SOURCES". "STOP"].
```

JSON

```
  ],
  "next_recommended_queries": [],
  "questions_to_ask": [
    {
      "slot": "sources_available",
      "prompt": "I don't have access to Slack/Email/Calendar/Docs here. Ca
      "options": []
    }
  ],
  "notes": [
    "Guideline prefers lots of links from internal sources; without access
    "Publishing without provenance violates no-invention guardrail."
  ]
}
```

---

3) Why this balances flexibility and controllability

- **Not rigid:** It doesn't predefine turn-by-turn dialogue; it constrains by **slots, budgets, checkpoints, and allowed acts**.

- **Still controllable:** It enforces:

JSON

  - minimum evidence targets (medium),
  - structural expectations (~20–25 bullets, 1–2 sentences) as revision guardrails (soft). GitHub

---

If you want, I can also produce:

- an **InteractionPolicy Rego** module that implements the checkpoint decisions (require_input vs allow vs require_tool_query),
- plus a **tool binding profile** for Slack/Email/Calendar/Docs that maps each `tool_recipe` to concrete connector queries (Gmail search operators, Slack query patterns, etc.).

Sources

Produce the InteractionPolicy Rego module and the tool binding profile as suggested

Below are two concrete artifacts:

1. **InteractionPolicy Rego module** that decides (at a checkpoint/turn) whether to **allow drafting**, **require input**, **require tool queries**, **require review**, or **deny publish**, and returns:
- `allowed_acts`
- `required_evidence`
- `next_recommended_queries` (recipe IDs + params)
- `questions_to_ask`
2. A **tool binding profile** that maps each `tool_recipe` to concrete connector/tool calls (Slack/Gmail/GCal/Docs/Web). I'll keep it "pluggable": you can swap the actual connector implementations without changing the recipe semantics.

---

1) InteractionPolicy Rego module

Assumptions about `input` shape (you can adapt as needed):

- `input.stage` : `"gather_sources" | "draft" | "publish"`

- `input.slots` : current slot values (e.g., `time_window` , `sections_strategy` , etc.)
- `input.capabilities` : booleans like `slack` , `email` , `calendar` , `docs` , `external_press`
- `input.evidence` : aggregated evidence stats and provenance checks, e.g.
  - `links_count` (int)
  - `all_links_and_claims_have_provenance` (bool)
- `input.budgets` : question budgets, etc.
- `input.params` : helper parameters like `company_name` , `exec_senders` list, channel hints.

```
package interaction.internal_comms.company_newsletter

default decision := "require_input"

# ----------------------------
# Helpers
# ----------------------------

missing_slot(name) {
  not input.slots[name]
}

# time_window required for gather_sources + draft
missing_time_window {
  missing_slot("time_window")
}

# "No invention" hard guardrail: must have provenance for links/claims bef
no_invention_ok {
  input.evidence.all_links_and_claims_have_provenance == true
}

# Links requirement (medium evidence requirement)
links_ok {
  input.evidence.links_count >= 8
}

# Connector availability
has_slack { input.capabilities.slack == true }
has_email { input.capabilities.email == true }
has_calendar { input.capabilities.calendar == true }
has_docs { input.capabilities.docs == true }
has_press { input.capabilities.external_press == true }

has_any_internal_source {
  has_slack or has_email or has_calendar or has_docs
```

```
}

# Produce allowed acts based on decision
allowed_acts := acts {
  decision == "require_input"
  acts := ["ASK_SLOT", "ASK_CHOICE", "STOP"]
} else := acts {
  decision == "require_tool_query"
  acts := ["REQUEST_SOURCES", "ASK_SLOT", "STOP"]
} else := acts {
  decision == "require_review"
  acts := ["ASK_SLOT", "REQUEST_SOURCES", "STOP"]
} else := acts {
  decision == "deny"
  acts := ["STOP"]
} else := acts {
  decision == "allow"
  acts := ["PROPOSE_DRAFT", "REVISE_DRAFT", "ASK_SLOT", "REQUEST_SOURCES"]
}

# ----------------------------
# Questions to ask (slot-driven)
# ----------------------------

questions[q] {
  # Ask time window first if missing
  missing_time_window
  q := {
    "slot": "time_window",
    "prompt": "Is this newsletter covering the past week, the past month,
    "options": ["past week", "past month", "custom range"]
  }
}

questions[q] {
  # If time_window present, optionally ask must-include topics
  not missing_time_window
  missing_slot("must_include_topics")
  q := {
    "slot": "must_include_topics",
    "prompt": "Any must-include announcements or topics you already know s
    "options": []
  }
}

questions[q] {
  # If sections strategy missing, ask at draft time
  input.stage == "draft"
  missing_slot("sections_strategy")
  q := {
    "slot": "sections_strategy",
    "prompt": "Prefer default sections (Announcements/Progress/Leadership/
```

```
    "options": ["default", "custom clusters"]
  }
}

# Budget: max 2 questions per turn
trimmed_questions := qs {
  all := [q | questions[q]]
  qs := array.slice(all, 0, 2)
}

# ----------------------------
# Recommended tool queries (recipes)
# ----------------------------

recommended_queries[r] {
  decision == "require_tool_query"
  has_slack
  r := {
    "recipe_id": "q.slack.high_signal_announcements",
    "params": {
      "time_window": input.slots.time_window,
      "channels_hint": input.params.channels_hint
    }
  }
}

recommended_queries[r] {
  decision == "require_tool_query"
  has_email
  r := {
    "recipe_id": "q.email.exec_companywide",
    "params": {
      "time_window": input.slots.time_window,
      "exec_senders_hint": input.params.exec_senders
    }
  }
}

recommended_queries[r] {
  decision == "require_tool_query"
  has_calendar
  r := {
    "recipe_id": "q.calendar.large_meetings",
    "params": {
      "time_window": input.slots.time_window
    }
  }
}

recommended_queries[r] {
  decision == "require_tool_query"
  has_docs
```

```
  r := {
    "recipe_id": "q.docs.recent_high_attention",
    "params": {
      "time_window": input.slots.time_window
    }
  }
}

recommended_queries[r] {
  decision == "allow"
  has_press
  # press query can be optional even when drafting is allowed
  r := {
    "recipe_id": "q.press.recent_mentions",
    "params": {
      "time_window": input.slots.time_window,
      "company_name": input.params.company_name
    }
  }
}

# ----------------------------
# Required evidence (by stage)
# ----------------------------

required_evidence[req] {
  input.stage == "draft" or input.stage == "publish"
  req := {"type": "provenance", "constraint": "no invented links or claims
}

required_evidence[req] {
  input.stage == "draft" or input.stage == "publish"
  req := {"type": "links", "constraint": ">=8 links across internal/extern
}

# ----------------------------
# Decision logic
# ----------------------------

# Deny publish if provenance violated (hard guardrail)
decision := "deny" {
  input.stage == "publish"
  not no_invention_ok
}

# Require input if time window missing (can’t proceed to sources/draft)
decision := "require_input" {
  missing_time_window
}

# Require tool query when drafting or gathering and links insufficient and
decision := "require_tool_query" {
```

```
  (input.stage == "gather_sources" or input.stage == "draft")
  not links_ok
  has_any_internal_source
}

# If tools not available and links insufficient, require review + ask user
decision := "require_review" {
  (input.stage == "gather_sources" or input.stage == "draft" or input.stag
  not links_ok
  not has_any_internal_source
}

# Allow drafting if time window is set and provenance OK (and either links
decision := "allow" {
  input.stage == "draft"
  not missing_time_window
  no_invention_ok
  links_ok
}

# Allow publish if provenance ok and links ok
decision := "allow" {
  input.stage == "publish"
  no_invention_ok
  links_ok
}

# Default is require_input (set above)
# ----------------------------
# Output object (single entry point)
# ----------------------------

output := {
  "decision": decision,
  "allowed_acts": allowed_acts,
  "required_evidence": [e | required_evidence[e]],
  "next_recommended_queries": [q | recommended_queries[q]],
  "questions_to_ask": trimmed_questions
}
```

How you'd call this policy

Your Policy Decision API can evaluate:

- `data.interaction.internal_comms.company_newsletter.output` with `input` being the current slot/evidence/capability state.

---

2) Tool binding profile

This profile maps the recipe IDs (from the InteractionIR) into actual connector calls. I'll express it as YAML. You can implement each binding in your runtime as "execute tool call + normalize results into evidence objects".

Notes:

- Slack/Gmail/GCal/Docs connectors may differ by platform. The binding specifies **what to call**, **how to parameterize**, and **how to normalize evidence**.
- For external press, use your web search tool.

```yaml
binding_profile:
  id: env.connectors.v1
  description: Tool bindings for internal-comms company newsletter recipes

  recipes:
    q.slack.high_signal_announcements:
      tool: slack.search
      args:
        # Example Slack search query format; adjust to your Slack connecto
        query: "in:(#announcements OR #company OR #all-hands) after:{start
        limit: 50
      param_mapping:
        start: "time_window.start"
        end: "time_window.end"
      postprocess:
        ranking:
          - field: reactions_count
            order: desc
          - field: thread_replies_count
            order: desc
        take_top: 10
      normalize_to_evidence:
        type: ev.slack_message
        fields:
          channel: "$.channel"
          message_url: "$.permalink"
          author: "$.user"
          timestamp: "$.timestamp"
          reactions_count: "$.reactions_count"
          thread_replies_count: "$.thread_replies_count"
          snippet: "$.text_snippet"

    q.email.exec_companywide:
      tool: gmail.search
      args:
        # Gmail search operators are standard; connector should accept raw
        query: "({exec_senders}) newer_than:{days}d"
```

```
      max_results: 25
    param_mapping:
      exec_senders: "params.exec_senders_hint"   # list -> OR-joined str
      days: "time_window.days"                   # runtime computes days
    postprocess:
      ranking:
        - field: date
          order: desc
      take_top: 10
    normalize_to_evidence:
      type: ev.email
      fields:
        from: "$.from"
        subject: "$.subject"
        date: "$.date"
        message_url_or_id: "$.id"                # use display_url if co
        snippet: "$.snippet"

  q.calendar.large_meetings:
    tool: gcal.search_events
    args:
      time_min: "{start_iso}"
      time_max: "{end_iso}"
      query: "All Hands OR Town Hall OR Company Update"
      max_results: 50
    param_mapping:
      start_iso: "time_window.start_iso"
      end_iso: "time_window.end_iso"
    postprocess:
      # If connector returns attendees_count, prefer large meetings; els
      filter:
        - expr: "($.attendees_count == null) OR ($.attendees_count >= 20
      ranking:
        - field: attendees_count
          order: desc
        - field: start_time
          order: desc
      take_top: 5
    normalize_to_evidence:
      type: ev.calendar_event
      fields:
        title: "$.title"
        date: "$.start_time"
        attendee_count: "$.attendees_count"
        description_snippet: "$.description_snippet"
        attached_docs: "$.attachments"

  q.docs.recent_high_attention:
    tool: docs.search
    args:
      # Replace with your actual docs connector; could be Google Drive s
      query: "modified_after:{start_minus_14d} OR created_after:{start_m
```

```
        max_results: 50
      param_mapping:
        start_minus_14d: "time_window.start_minus_days_14"
      postprocess:
        ranking:
          - field: attention_signal
            order: desc
          - field: modified_time
            order: desc
        take_top: 10
      normalize_to_evidence:
        type: ev.doc
        fields:
          title: "$.title"
          doc_url: "$.url"
          owner_or_author: "$.owner"
          published_date: "$.modified_time"
```

rego

```
    q.press.recent_mentions:
      tool: web.search
      args:
        query: "{company_name} press after:{start} before:{end}"
        recency_days: 30
        limit: 10
      param_mapping:
        company_name: "params.company_name"
        start: "time_window.start"
        end: "time_window.end"
      postprocess:
        ranking:
          - field: publisher_tier
            order: desc
          - field: date
            order: desc
        take_top: 3
      normalize_to_evidence:
        type: ev.press
        fields:
          title: "$.title"
          url: "$.url"
          publisher: "$.publisher"
          date: "$.date"

  evidence_aggregation:
    links_count:
      compute: "count(unique(extract_links(evidence_items)))"
    all_links_and_claims_have_provenance:
      compute: >
        enforce that any link in draft appears in evidence_items or user_p
        any nontrivial factual claim must cite an evidence_item id or be m
```

What your runtime needs to implement these bindings

- A templating system for `query_template` / `{param}` interpolation.
- A normalizer that maps tool results into the declared evidence types.
- An evidence aggregator that computes:
  - `links_count`
  - `all_links_and_claims_have_provenance`
- Connector stubs:
  - `slack.search`, `gmail.search`, `gcal.search_events`, `docs.search`, `web.search`

If some connectors aren't available, set `input.capabilities.<source>=false` and the Rego policy will route to "require_review" + ask user to provide sources.

If you want, I can also generate:

- a **Decision API OpenAPI schema** specifically for Interaction decisions (stage/slots/evidence/capabilities input; decision/acts/queries output), and
- a **minimal "evidence provenance checker" algorithm** (how to verify "no invented links/claims" in a draft deterministically or semi-deterministically).